

算法设计

Design of Algorithms

第1章 算法概述

本章内容

1.1 算法概述

- 算法来源与概念
- 程序与算法的区别和联系？
- 算法与数据结构的关系？
- 算法应用

1.2 学习方法

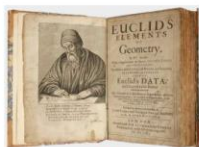
- 授课方式
- 考核方式
- 教材与资源

1.3 计算复杂性

- 计算复杂性概念
- 算法渐近复杂性分析方法及应用
- 描述算法的方法

1.1 算法概述 算法来源与概念

算法的由来

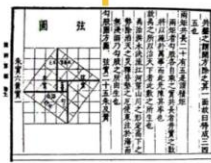


公元前300年
辗转相除法
《几何原本》



7世纪
《算经十书》

算法
《周髀算经》
公元前1世纪



Algorithm
阿尔·花拉子密
9世纪



魏晋时期
割圆术
《九章算术注》



20世纪30年代~40年代
艾伦·图灵与冯·诺依曼

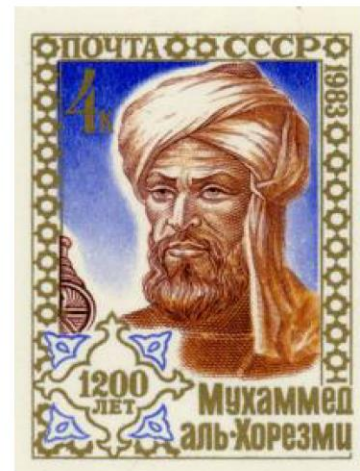
1.1 算法概述 算法来源与概念

算法的由来



- 名称由来

- 波斯著名的数学家、天文学家、地理学家阿尔·花拉子密在公元825年写成《印度数字算术》一书，对于印度-阿拉伯数字系统在中东及欧洲的传播起到了重要作用
- 该书被翻译成拉丁语 “Algoritmi de numero Indorum”，花拉子密的拉丁文音译即为 “算法”（Algorithm）一词的由来



苏联在1983年发行邮票纪念花拉子密1200岁生辰

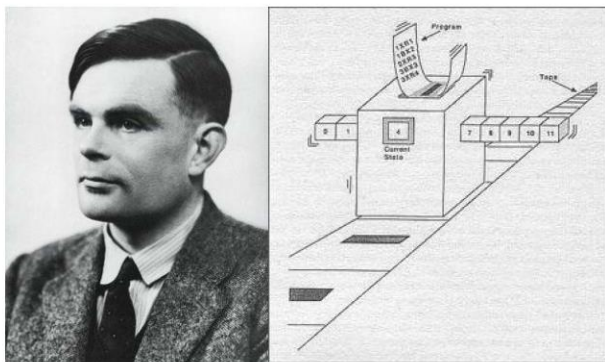
1.1 算法概述 算法来源与概念

算法的由来

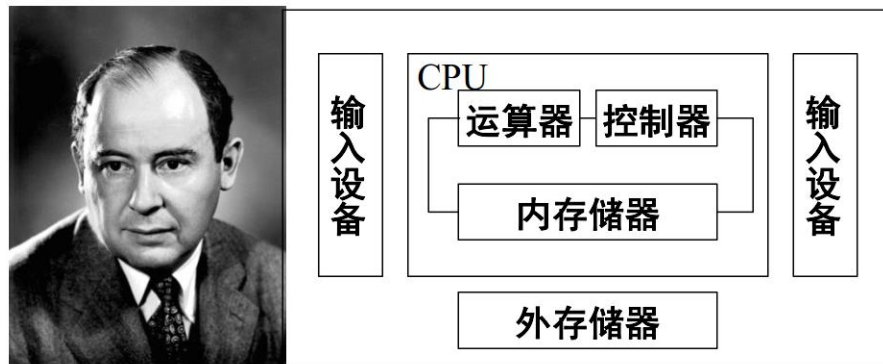


● 算法与计算机的结合

- 1936年，艾伦·图灵提出**图灵机**，通过建立通用计算机模型，刻画计算机的计算行为
- 1946年，冯·诺依曼提出**存储程序原理**



理论计算机科学与人工智能之父
艾伦·图灵
Alan Turing



现代计算机之父
约翰·冯·诺伊曼
John von Neumann

1.1 算法概述 算法来源与概念

算法与计算复杂性领域图灵奖得主



Donald E. Knuth
1974, USA
算法分析之父



Michael O. Rabin
1976, Israeli
非确定自动机
素数判定随机算法



Dana S. Scott
1976, USA
非确定自动机



Robert W. Floyd
1978, USA
最短路径Floyd算法



Stephen A. Cook
1982, USA
NP完全性



Richard M. Karp
1985, USA
NP完全性与
网络流算法



John Hopcroft
1986, USA
最差情况分析
数据结构与算法



Robert Tarjan
1986, USA
数据结构与图算法



Juris Hartmanis
1993, Latvia
计算复杂性理论



Richard E. Stearns
1993, USA
计算复杂性理论



Manuel Blum
1995, Venezuela
计算复杂性理论



Andrew Yao
2000, China
伪随机数生成
与通信复杂性

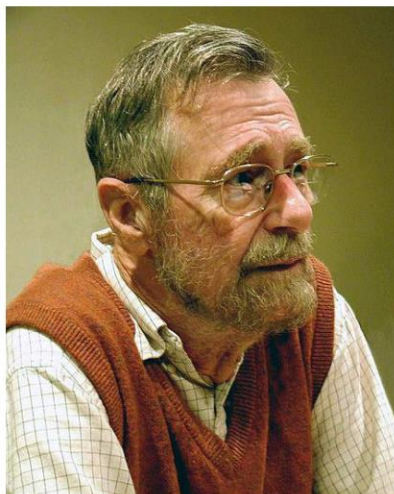


Leslie G. Valiant
2010, Hungarian
#P完全性与
计算学习理论

算法与计算复杂性领域图灵奖得主共计13人，占比约18.6%

1.1 算法概述 算法来源与概念

其他相关图灵奖得主



Edsger W. Dijkstra
1972, Netherlands
ALGOL之父
提出单源最短路径Dijkstra算法



Tony Hoare
1980, UK
霍尔逻辑
提出快速排序算法

1.1 算法概述 算法来源与概念

- **算法**：解决问题的一种方法或一个过程。

- **性质**：

- (1) **输入**：有外部提供的一组变量作为算法的输入。有的算法表面上可以没有输入，实际上已被嵌入算法之中。

- (2) **输出**：算法产生至少一个量作为输出，一组输入信息加工后得到的结果。

- (3) **确定性**：组成算法的每条指令是清晰、无歧义的。在任何条件下，算法都只有一条执行路径。

- (4) **有限性**：算法中每条指令的执行次数是有限的，每条指令的执行时间也是有限的。

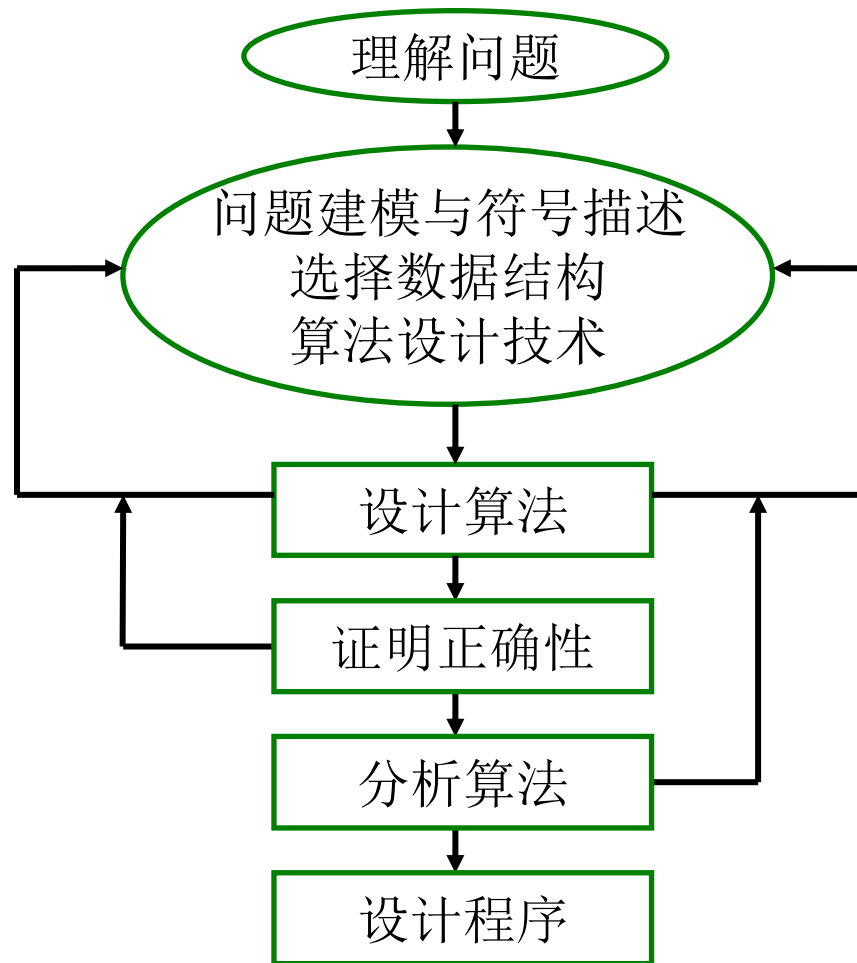
1.1 算法概述 程序与算法的区别和联系

- 程序：算法用某种程序设计语言的具体实现。
- 程序可以不满足算法的性质(4)有限性。
 - 例如操作系统，是一个在无限循环中执行的程序，因而不是一个算法。
- 操作系统的各种任务可看成是单独的问题，每一个问题由操作系统中的一个子程序，通过特定的算法来实现。
该子程序得到输出结果后便终止。

1.1 算法概述 算法与数据结构的关系

- 计算机科学家 N.Wirth 教授：程序 = 数据结构 + 算法
- 数据结构是数据间的有机关系，算法是对数据的操作步骤。
- 有了数据结构，算法才能诞生。反之，算法又是数据结构得以维持的一个条件。
- 二者不可分割。没有一定组织关系的数据，算法就无法运行。

1.1 算法概述 问题求解



1.1 算法概述 算法应用

➤ 算法应用于各种应用系统中

Internet. Web search, packet routing, distributed sharing.

Biology. Human genome project, protein folding

Computers. Circuit layout, file system, com

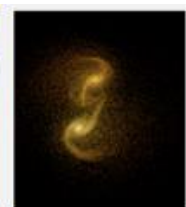
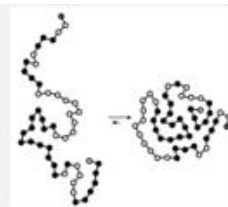
Computer graphics. Moveies. Video games, virtual reality

Security. Cell phones, e-commerce, voting machines

Multimedia. MPS, JPG, face recognition

Social networks. Recommendations, news feeds, advertisements

Physics. N-body simulation, particle collision simulation



1.2 学习方法

➤ 授课方式

- 讲授、课堂练习、上机实验

➤ 考核方式

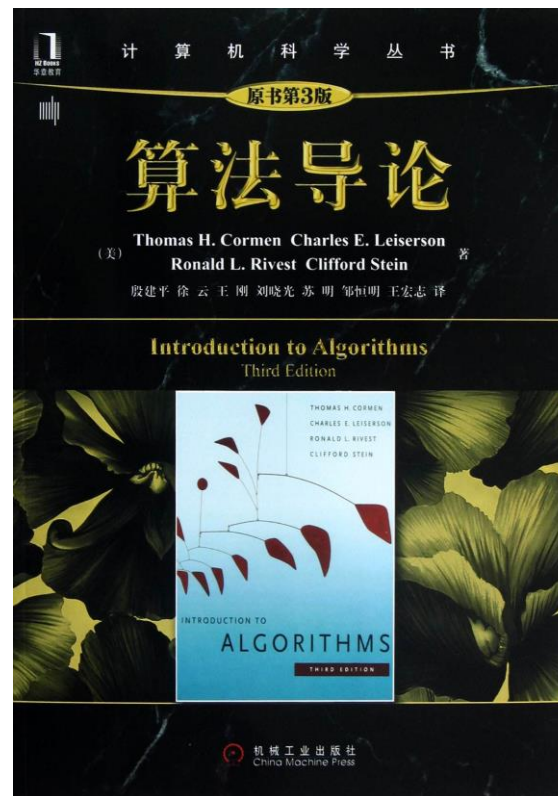
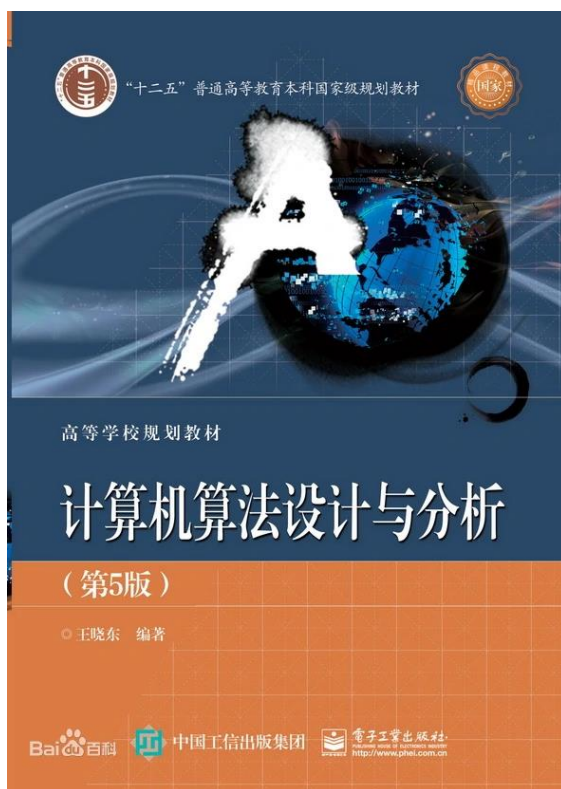
- 考试课，平时成绩（课堂表现、作业、课堂算法编写）、期末考试

➤ 教材与资源

参考书籍

计算机算法设计与分析（第五版） 作者：王晓东

算法导论（第3版）



参考课程

算法设计与分析 东北大学

https://www.bilibili.com/video/BV17Y4y187Nr?spm_id_from=333.999.0.0

- 算法设计与分析 MOOC 北京航空航天大学

<https://www.icourse163.org/course/BUAA-1449777166>

代码随想录 Carl

https://www.bilibili.com/video/BV1BU4y177kY?spm_id_from=333.337.search-card.all.click

算法设计与分析

<https://www.bilibili.com/video/BV1wi4y1g7PR?p=3>

【北大公开课】 算法设计与分析

<https://www.bilibili.com/video/BV1Ls411W7PB?p=1>

战疫时期的算法课-南京大学-2020年春季

https://www.bilibili.com/video/BV11341167sn/?p=2&spm_id_from=pageDriver

1.3 算法复杂性分析

- 目的：比较求解同一问题的多个不同算法的好坏，时空耗费；
- 算法复杂性：算法运行所需的计算机资源的量
- 算法分析：一个算法所消耗资源进行估算，包括时间、空间；
- 算法的时间复杂性 $T(n)$ —所需的时间资源的量，其中 n 是问题的规模（输入大小）
- 算法的空间复杂性 $S(n)$ —所需的空间资源的量
- 主要讨论时间复杂性，空间复杂性分析相对比较简单，计量方法相似。

1.3 算法复杂性分析

- (1) 最坏情况下的时间复杂性

$$T_{\max}(n) = \max\{ T(i) \mid \text{size}(i)=n \},$$

规模为 n 的所有输入实例中运行时间最长的实例的运行时间。可操作性最好，最有实际价值，本书重点。

- (2) 最好情况下的时间复杂性

$$T_{\min}(n) = \min\{ T(i) \mid \text{size}(i)=n \}$$

规模为 n 的所有输入实例中运行时间最短的实例运行时间。

- (3) 平均情况下的时间复杂性

$$T_{\text{avg}}(n) = \sum_{\text{size}(I)=n} p(I)T(I)$$

规模为 n 的所有输入实例运行时间的概率加权和。其中 I 是问题的规模为 n 的实例， $p(I)$ 是实例 I 出现的概率。

1.3 算法复杂性分析

时间耗费分析(最好、最坏、平均情况)

例如：在一维数组结构上的查找，一旦查找成功算法结束。输入 n 个不同的数据，要查找的数据为 k （假设 k 在 n 个不同的数据中）。

- 问题规模：数组长度 n
- 基本操作：元素的比较
- 算法策略：顺序查找法，依次在数组中取出每个元素与 k 比较，对于成功的查找，有可能第一个元素就是要查找的 k ，元素的比较次数为1次，这是算法的最佳情况。

1.3 算法复杂性分析

时间耗费分析(最好、最坏、平均情况)

- 如果第 n 个元素才是要查找的 k , 那么, 元素间的比较要进行 n 次, 这是算法的最差情况。
- 一般地, 对于成功的比较, 假定 k 出现在第 i 个位置上, 是等概率的; $0 < i < n+1$. 元素的比较平均要进行 $(n+1)/2$ 次, 这是算法的时间耗费的平均情况。

1.3 算法复杂性分析

- **最好情况**不具有普遍性，**最坏情况**表明时间耗费的上界是多少，而平均情况更具有普遍性。
- 但是对于**时间耗费与输入有关**的算法来说，要求得它的平均性能并不那么容易，有些算法早就有人提出，但是至今在数学上也没能求得它的平均性能的值。
- 实际应用中最为关注的是**最差情况的算法分析**。
- 在非专门说明时间分析是指平均情况的时间耗费，一般情况下将**最坏情况下的时间耗费的极限作为算法的时间耗费**，称为时间复杂性。
- 求解过程称为时间渐近分析。

1.3 算法复杂性分析

算法渐近复杂性

- **单调递增**：当 $n \rightarrow \infty$ 时， $T(n) \rightarrow \infty$ ，则称 $T(n)$ 单调递增的；
- **算法的渐近复杂性**：对于 $T(n)$ ，如果存在 $t(n)$ ，当 $n \rightarrow \infty$ 时， $(T(n)-t(n))/T(n) \rightarrow 0$ ，称 $t(n)$ 是 $T(n)$ 的渐近性态，或为算法的 **渐近复杂性**。
- 在数学上， $t(n)$ 是 $T(n)$ 的渐近表达式，是 $T(n)$ 略去低阶项留下的主项。它比 $T(n)$ 简单。
- 为什么当 $n \rightarrow \infty$ 可以用算法渐近复杂性替代表示算法复杂性？
- (1) 当 $n \rightarrow \infty$ ， $t(n)/T(n)=1$
- (2) 渐近表达式 $t(n)$ 是算法复杂性 $T(n)$ 略去低阶项留下的主项，简化了 $T(n)$
- **目的**：简化复杂性分析，通过算法渐近复杂性分析算法复杂性，更简单。

算法复杂性分析进一步简化

对于要比较的两个算法渐近复杂性的阶不同时：

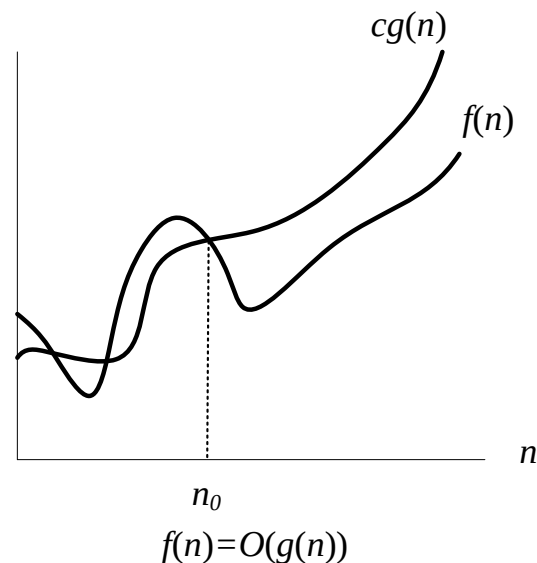
- 考虑到分析算法复杂性的目的：**比较求解同一问题的两个不同算法的效率**。因此，只要得到不同算法的渐近复杂性 $t(n)$ 的阶即可判定哪一个算法的效率更高。
- 不必关心算法的渐近复杂性 $t(n)$ 中的常数因子。
- 为什么可以这样简化？因为这时算法复杂性的高低取决于算法渐近复杂性的阶，与 $t(n)$ 中的常数因子无关。例如 $f(n) = c_1 n^2$ ， $g(n) = c_2 n^3$ ，因为当 $n \rightarrow \infty$ ， $f(n) / g(n) \rightarrow 0$ 。当 $n \rightarrow \infty$ ， $f(n) < g(n)$ 。

渐近分析的记号

在下面的讨论中, 对所有 n , $f(n) \geq 0$,
 $g(n) \geq 0$ 。

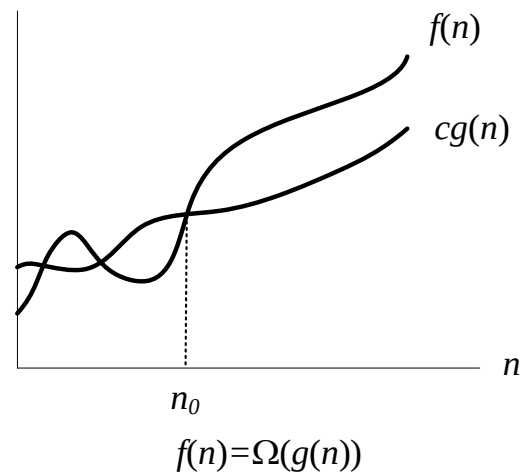
(1) 渐近上界记号 O

$O(g(n)) = \{ f(n) \mid \text{存在正常数 } c \text{ 和 } n_0 \text{ 使得对}$
所有 $n \geq n_0$ 有: $0 \leq f(n) \leq cg(n) \}$



(2) 渐近下界记号 Ω

$\Omega(g(n)) = \{ f(n) \mid \text{存在正常数 } c \text{ 和 } n_0 \text{ 使得对}$
所有 $n \geq n_0$ 有: $0 \leq cg(n) \leq f(n) \}$



渐近分析的记号

(3) 非紧上界记号 o

$o(g(n)) = \{ f(n) \mid \text{对于任何正常数 } c > 0, \text{ 存在正数和 } n_0 > 0 \text{ 使得对所有 } n \geq n_0 \text{ 有: } 0 \leq f(n) < cg(n) \}$

等价于 $f(n) / g(n) \rightarrow 0$, as $n \rightarrow \infty$ 。

(4) 非紧下界记号 ω

$\omega(g(n)) = \{ f(n) \mid \text{对于任何正常数 } c > 0, \text{ 存在正数和 } n_0 > 0 \text{ 使得对所有 } n \geq n_0 \text{ 有: } 0 \leq cg(n) < f(n) \}$

等价于 $f(n) / g(n) \rightarrow \infty$, as $n \rightarrow \infty$ 。

$$f(n) \in \omega(g(n)) \Leftrightarrow g(n) \in o(f(n))$$

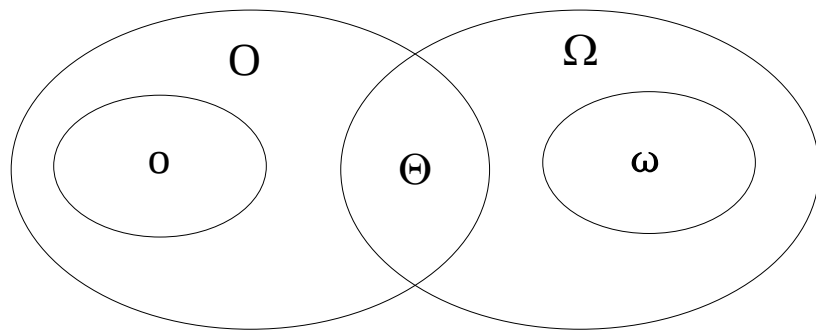
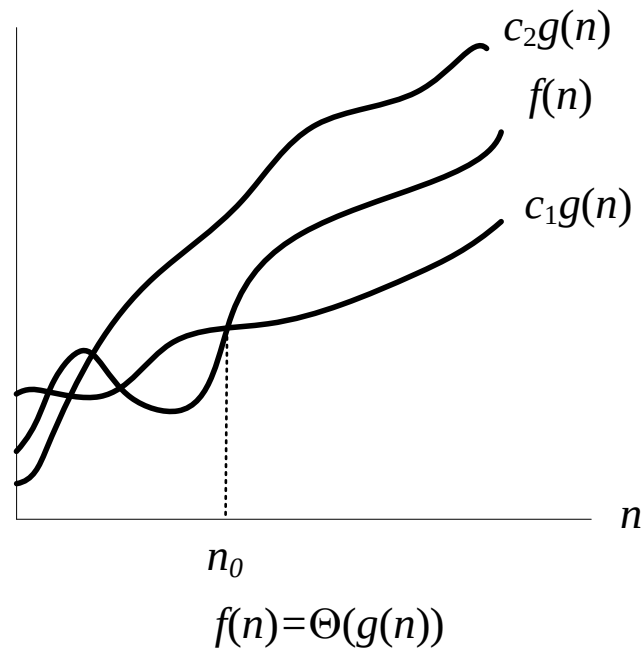
渐近分析的记号

(5) 紧渐近界记号 Θ

$\Theta(g(n)) = \{ f(n) \mid \text{存在正常数 } c_1, c_2 \text{ 和 } n_0$
使得对所有 $n \geq n_0$ 有: $c_1 g(n) \leq f(n) \leq$
 $c_2 g(n) \}$

定理1: $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

- o 与 ω 无明确的实际意义;
- O 最常用, 通用
- Θ 最准确, 能求 Θ 尽量用。



渐近分析中函数比较

- $f(n) = O(g(n)) \approx a \leq b$;
- $f(n) = \Omega(g(n)) \approx a \geq b$;
- $f(n) = \Theta(g(n)) \approx a = b$;
- $f(n) = o(g(n)) \approx a < b$;
- $f(n) = \omega(g(n)) \approx a > b$.

渐近分析记号的若干性质

- (1) 传递性:

- $f(n) = \Theta(g(n)), \quad g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n));$

- $f(n) = O(g(n)), \quad g(n) = O(h(n)) \Rightarrow f(n) = O(h(n));$

- $f(n) = \Omega(g(n)), \quad g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n));$

- $f(n) = o(g(n)), \quad g(n) = o(h(n)) \Rightarrow f(n) = o(h(n));$

- $f(n) = \omega(g(n)), \quad g(n) = \omega(h(n)) \Rightarrow f(n) = \omega(h(n));$

渐近分析记号的若干性质

- (2) 反身性:

- $f(n) = \Theta(f(n));$

- $f(n) = O(f(n));$

- $f(n) = \Omega(f(n)).$

- (3) 对称性:

- $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n)).$

- (4) 互对称性:

- $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n));$

- $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n));$

渐近分析记号的若干性质

- (5) 算术运算:

- $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\}) ;$

- $O(f(n)) + O(g(n)) = O(f(n) + g(n)) ;$

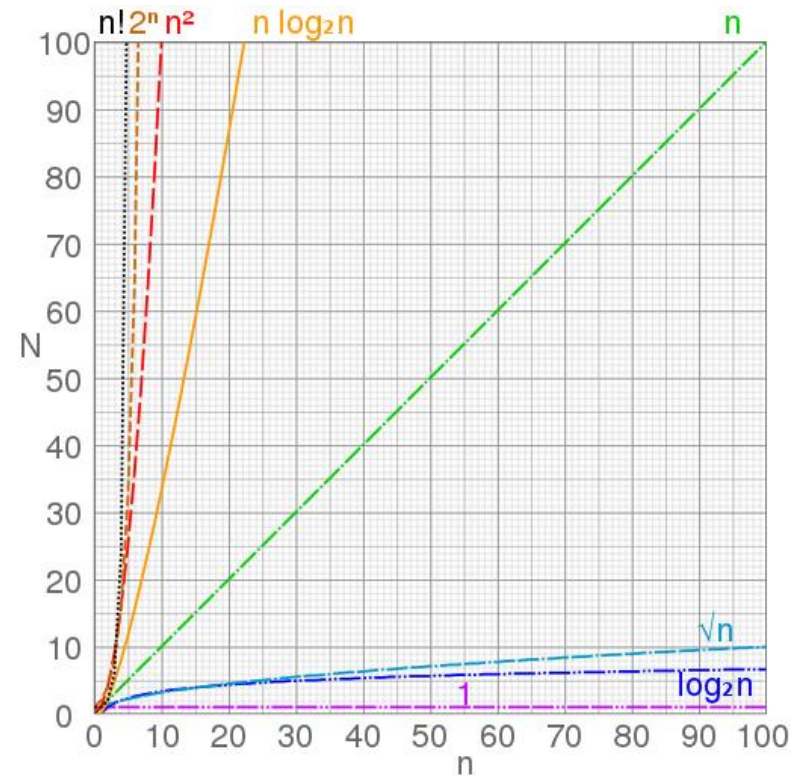
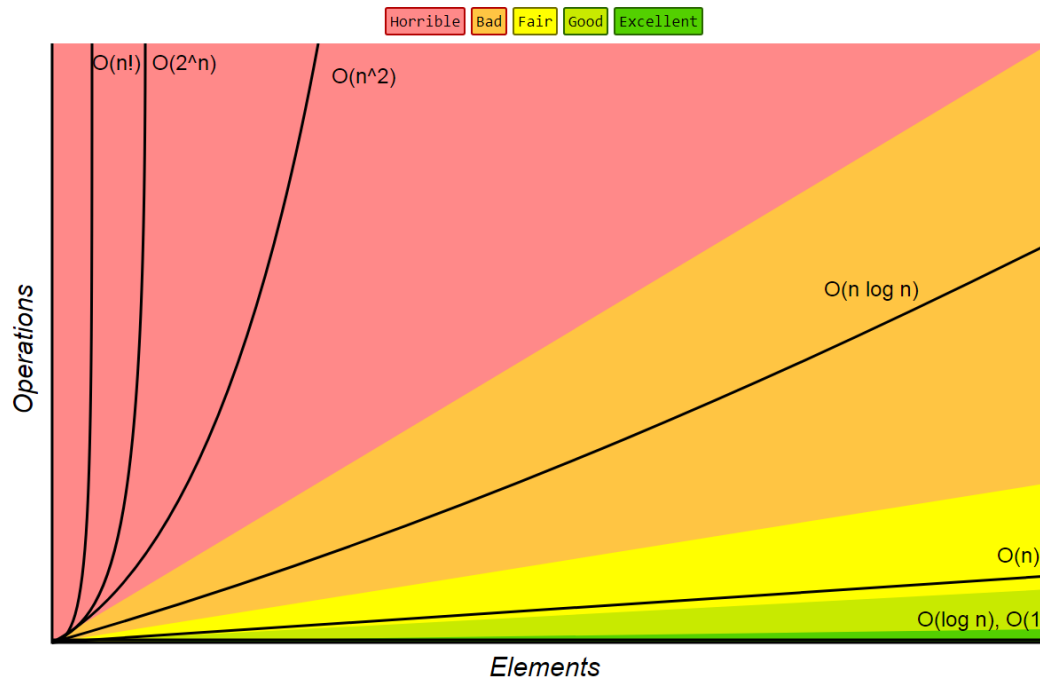
- $O(f(n)) * O(g(n)) = O(f(n) * g(n)) ;$

- $O(cf(n)) = O(f(n)) ;$

- $g(n) = O(f(n)) \Rightarrow O(f(n)) + O(g(n)) = O(f(n)) .$

$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65536
5	32	160	1024	32768	4294967296

Big-O Complexity Chart



$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

算法分析方法

三种表示法（代换法，递归树法，主定理法）

- 代换法：猜测有某个界存在（上界或者下界），然后用数学归纳法证明. 不好猜。
- 递归树法：将递归式转换成树形结构，树的节点代表在不同递归层次付出的代价，最后利用对和式限界的技术来求解递归式。
（数学对数的运算，求出后要用代换法证明之）
- 主定理法:

Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

练习1.1： 按阶排列表达式。

按渐近阶从低到高的顺序排列以下表达式：

$4n^2$, $\log n$, 3^n , $20n$, $n!$, 2 , $n^{2/3}$

练习1.2：硬件效率。

硬件厂商A公司宣称他们最新研制的微处理器运行速度为其竞争对手B公司同类产品的100倍。对于计算复杂性分别为 n, n^2, n^3 的各算法，若用B公司的计算机在1小时内能解输入规模为 n 的问题，那么用A公司的计算机在1小时内分别能解输入规模为多大的问题。